

Capítulo 7: Manejo de Ausencia y Errores Puros

1. Explicación Teórica: El Error como un Valor

El manejo de errores en Scala se alinea con los principios de la Programación Funcional (PF), donde las funciones ideales deben ser **puras** y carecer de **efectos secundarios**. Una consecuencia de esta filosofía es que los fallos, la ausencia de valores y los errores no deben interrumpir el flujo de control (como lo hacen las excepciones `throw`) ni introducir ambigüedad (como lo hace `null`).

En su lugar, Scala modela tanto la ausencia como el error como **valores de datos inmutables** que se encapsulan en estructuras de contenedores. Esto permite que los errores y los valores opcionales sean tratados como cualquier otro dato, haciendo que el código sea más seguro y componible.

A. Ausencia de Valor (`Option[T]`)

El tipo `Option[T]` es la solución idiomática de Scala para el problema de la **referencia nula** (`null`), que es la causa principal de las famosas `NullPointerException`. `Option[T]` es un contenedor de valor seguro que garantiza que el desarrollador maneje la posible ausencia de un resultado.

- **Composición:** Funciona como una "colección" que contiene cero o un elemento. Por lo tanto, se puede transformar utilizando Funciones de Orden Superior (HOFs) como `map` y `flatMap`.

B. Manejo de Errores Puros (`Try[T]` y `Either[E, R]`)

En lugar de depender de bloques `try/catch` que detienen la ejecución, Scala proporciona contenedores que envuelven el resultado de una operación que puede fallar.

- `Try[T]` : Se utiliza principalmente para **capturar excepciones** existentes en código que interactúa con el mundo externo (I/O, librerías Java). Un `Try` resultará en un `Success(valor)` si la operación fue exitosa, o un `Failure(excepción)` si se lanzó una excepción.
- `Either[E, R]` : Es la estructura más flexible y preferida en el código funcional puro. `Either` tiene dos parámetros de tipo: `E` para la izquierda (Error) y `R` para la derecha (Resultado o Éxito). Este tipo fuerza al desarrollador a **tipar el error** de forma explícita (a diferencia de `Try`, que solo captura `Throwable`).

2. Sintaxis y Estructura Funcional

Tanto `Option`, `Try` como `Either` son **Tipos de Datos Algebraicos (ADTs)**, lo que significa que son tipos de suma (`Sum Types`) que encapsulan sus posibles estados internos.

A. Sintaxis de Option

`Option[T]` es un tipo base con dos subtipos (variantes) principales:

Variante	Descripción	Sintaxis
<code>Some[T]</code>	Indica que un valor está presente.	<code>val resultado = Some("Valor encontrado")</code>
<code>None</code>	Objeto <i>Singleton</i> que indica la ausencia del valor.	<code>val ausente: Option[String] = None</code>

B. Sintaxis de Either

`Either[E, R]` es un contenedor *sesgado a la derecha* por convención, lo que significa que el valor de éxito se coloca en el lado `Right` y el valor de error en el lado `Left`.

Variante	Descripción	Sintaxis
<code>Right[R]</code>	Indica éxito. <code>R</code> es el valor de resultado.	<code>val exito = Right(42)</code>
<code>Left[E]</code>	Indica fallo. <code>E</code> es el valor o mensaje de error tipado.	<code>val fallo = Left("Conexión perdida")</code>

El uso de `map`, `flatMap` o *for comprehensions* en estos tipos permite encadenar operaciones sin *unwrapping* (desenvolver) manualmente el contenedor ni verificar `null`. Si el contenedor es `None` o `Left`, el encadenamiento simplemente se salta la operación y propaga el estado de fallo.

3. Ejemplos de Código (Realistas)

A. Uso de Option : Manejo Seguro de una Búsqueda en Map

Buscar una clave en un `Map` es un caso clásico donde el resultado es incierto. El método `get` de `Map` devuelve automáticamente un `Option[V]`.

```
1 // Simula un Map de configuración inmutable (clave String -> valor Int)
2 val configuraciones = Map("timeout_ms" -> 5000, "retries" -> 3)
3
4 // 1. Acceso Seguro (devuelve Option[Int])
5 val timeoutOpt: Option[Int] = configuraciones.get("timeout_ms")
6 val portOpt: Option[Int] = configuraciones.get("port")
```

```

7
8 // 2. Composición funcional (Happy Path): usa map para duplicar el valor
  si existe
9 val nuevoTimeoutOpt = timeoutOpt.map(_ * 2)
10 // nuevoTimeoutOpt: Some(10000)
11
12 // 3. Manejo de ausencia con valor por defecto
13 val puertoFinal: Int =
14     portOpt.getOrElse(8080) // Si es None, usa 8080. Si es Some(v), devuelve
    v.
15 // puertoFinal: 8080
16
17 // 4. Pattern Matching (alternativa idiomática a if/else)
18 val descripcion = puertoFinal match
19     case 8080 => "Puerto por defecto"
20     case t if t > 0 => s"Puerto configurado: $t"
21     case _ => "Error de puerto"

```

B. Uso de `Either`: Conversión de String y Propagación de Fallo

Este ejemplo muestra cómo una función que intenta convertir una cadena en un número (una operación que puede fallar con una `NumberFormatException`) puede modelarse usando `Either`, lo que fuerza a tipar el error (`String`) y a manejarlo explícitamente.

```

1 // Definimos un ADT para tipificar los posibles errores (aunque un String
  simple también funciona)
2 enum ErrorDominio:
3     case FormatoInvalido(entrada: String)
4     case DatoFaltante
5
6 // Función que intenta convertir una String a Int, devolviendo
  Either[Error, Resultado]
7 def parseAndValidate(entrada: String): Either[ErrorDominio, Int] =
8     try {
9         val valor = entrada.toInt
10        if (valor <= 0)
11            Left(ErrorDominio.FormatoInvalido(entrada))
12        else
13            Right(valor) // Éxito: envuelto en Right
14    } catch {
15        case _: NumberFormatException =>
16            Left(ErrorDominio.FormatoInvalido(entrada)) // Fallo: envuelto en
        Left
17    }
18
19 val resultadoValido = parseAndValidate("123")

```

```

20 // Right(123)
21
22 val resultadoInvalido = parseAndValidate("abc")
23 // Left(FormatoInvalido("abc"))
24
25 // Encadenamiento funcional usando for comprehension (azúcar sintáctico
    para flatMap/map)
26 // La for comprehension se detiene si encuentra un Left (fallo)
27 val proceso = for {
28     num1 <- parseAndValidate("100") // Debe ser Right(100)
29     num2 <- parseAndValidate("10")  // Debe ser Right(10)
30 } yield num1 / num2
31 // proceso: Right(10)

```

4. Comparativa con Java y Python

Aspecto	Java (Imperativo)	Python (Dinámico)	Scala (Funcional/Puro)
Ausencia de Valor	<code>null</code> . Causa errores de tipo <code>NullPointerException</code> .	<code>None</code> o <code>null</code> .	<code>Option[T]</code> . Permite componer el código (<code>.map</code>) sin verificación explícita de nulidad.
Manejo de Errores	<code>throw</code> / <code>try-catch</code> . Interrumpe el flujo de ejecución (<code>checked/unchecked exceptions</code>).	<code>raise</code> / <code>try-except</code> .	<code>Either[E, R]</code> / <code>Try[T]</code> . El fallo es un valor que se propaga, no una interrupción.
Composición	Difícil encadenar operaciones que pueden fallar sin múltiples bloques <code>try-catch</code> .	Composición limitada en el manejo de errores.	Uso de <code>map</code> o <code>flatMap</code> sobre <code>Option</code> / <code>Either</code> para construir secuencias de operaciones de forma declarativa.
Tipado del Error	La mayoría son <code>Throwable</code> o <code>Exception</code> genéricos.	Errores basados en la clase.	<code>Either</code> permite tipar explícitamente el error (<code>E</code>) con un <code>String</code> , una <i>Case Class</i> o un <code>enum</code> .

5. Best Practices (*The Scala Way*)

- Eliminación de `null`**: Nunca se debe devolver o aceptar `null` en un código Scala idiomático. Se debe utilizar `Option[T]` en su lugar para modelar cualquier valor que

pueda estar ausente.

2. **Transparencia de Errores:** Evitar lanzar excepciones (`throw`) desde funciones puras o métodos que definen la lógica de negocio. En su lugar, el **error debe ser un valor** que se retorne dentro de un contenedor, garantizando la **transparencia referencial**.
3. **Elección del Contenedor:**
 - Usar `Try[T]` cuando el objetivo es interactuar con una API existente de Java que lanza excepciones que no se pueden modificar, ya que `Try` está diseñado para *capturar* `Throwable`.
 - Usar `Either[E, R]` para definir la lógica interna y APIs que requieren errores tipados específicos, ya que `Either` permite modelar `E` (Error) como una *case class* o *enum*.
4. **Composición Funcional:** Utilizar `map` y `flatMap` (o *for comprehensions*, que son azúcar sintáctico para estos) para encadenar operaciones sobre estos contenedores. Este enfoque permite al desarrollador enfocarse en el "camino feliz" (el flujo de éxito) y delegar la propagación de errores al sistema de tipos.